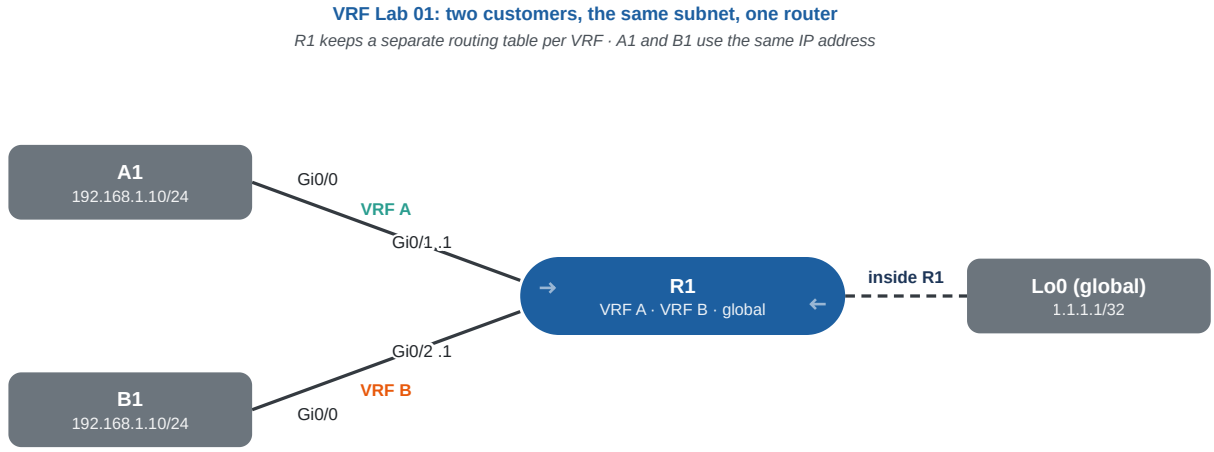
VRF Lab 01: VRF-Lite Basics

Lab Objective:

The focus of this lab is to understand Virtual Routing and Forwarding (VRF) on a single Cisco IOS router. Additional technologies include creating VRFs, assigning interfaces to them, overlapping IP addresses, per-VRF routing tables, VRF-aware ping, and static routes inside a VRF.

Lab Topology:

The lab network topology is illustrated below:



IMPORTANT NOTE

Use IOSv for R1, A1 and B1. A1 and B1 are customer routers acting as hosts, each with a default route to 192.168.1.1. Each also gets a Loopback1 to play a second network behind it: A1 Lo1 10.10.10.1/24, B1 Lo1 10.10.10.1/24 (the same, on purpose).

A **VRF** is a separate routing table inside one router. Interfaces in VRF A only see routes in VRF A. That's why two customers can use the same addresses on one box. Without MPLS this is called **VRF-Lite**.

Task 1

Configure R1's Loopback0 with 1.1.1.1/32 in the global table. Create VRFs **A** and **B** on R1 for IPv4. Configure A1 and B1 with their addresses, loopbacks and default routes.

Task 2

Put R1 Gi0/1 into VRF A and Gi0/2 into VRF B, both with the address 192.168.1.1/24. Note what happens when you assign the VRF to an interface that already has an IP address. Verify the VRFs and their interfaces.

Task 3

Verify the routing table of VRF A, VRF B and the global table. Ping A1 and B1 from R1. Explain why `ping 192.168.1.10` alone fails, and why R1 can ping the same address in two places.

Task 4

Make 10.10.10.0/24 behind A1 reachable from R1 in VRF A, and 10.10.10.0/24 behind B1 reachable in VRF B, with static routes. Verify that both work at the same time, and that neither customer can reach R1's Loopback0.

Lab Validation:

The outputs below are trimmed.

Task 1

```
R1(config)#interface Loopback0
R1(config-if)#ip address 1.1.1.1 255.255.255.255
R1(config-if)#exit
R1(config)#vrf definition A
R1(config-vrf)#address-family ipv4
R1(config-vrf-af)#exit-address-family
R1(config-vrf)#exit
R1(config)#vrf definition B
R1(config-vrf)#address-family ipv4
R1(config-vrf-af)#exit-address-family
```

```
A1(config)#interface GigabitEthernet0/0
A1(config-if)#ip address 192.168.1.10 255.255.255.0
A1(config-if)#no shutdown
A1(config-if)#interface Loopback1
A1(config-if)#ip address 10.10.10.1 255.255.255.0
A1(config-if)#exit
A1(config)#ip route 0.0.0.0 0.0.0.0 192.168.1.1
```

B1 gets exactly the same configuration.

`vrf definition` with `address-family ipv4` is the modern syntax and supports IPv4 and IPv6. The older `ip vrf A` (IPv4 only) still works and you'll see it in older configs and books. A **route distinguisher (rd)** isn't needed for VRF-Lite. It only matters with MPLS VPNs and BGP.

Task 2

```
R1(config)#interface GigabitEthernet0/1
R1(config-if)#ip address 192.168.1.1 255.255.255.0
R1(config-if)#vrf forwarding A
% Interface GigabitEthernet0/1 IPv4 disabled and address(es) removed due to enabling VRF A
R1(config-if)#ip address 192.168.1.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#interface GigabitEthernet0/2
R1(config-if)#vrf forwarding B
R1(config-if)#ip address 192.168.1.1 255.255.255.0
R1(config-if)#no shutdown
```

Assigning a VRF removes the interface's IP address, because the address has to move from the global table into the VRF's table. Always configure `vrf forwarding` first and the IP address second. With the older syntax the command is `ip vrf forwarding A`.

```
R1#show vrf
  Name                             Default RD            Protocols   Interfaces
  A                                <not set>             ipv4        Gi0/1
  B                                <not set>             ipv4        Gi0/2

R1#show ip vrf interfaces
Interface              IP-Address      VRF                              Protocol
Gi0/1                  192.168.1.1     A                                up
Gi0/2                  192.168.1.1     B                                up
```

The same address on two interfaces of one router is accepted, because each lives in a different table.

Task 3

```
R1#show ip route vrf A | begin Gateway
Gateway of last resort is not set
      192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.1.0/24 is directly connected, GigabitEthernet0/1
L        192.168.1.1/32 is directly connected, GigabitEthernet0/1

R1#show ip route | begin Gateway
Gateway of last resort is not set
      1.0.0.0/32 is subnetted, 1 subnets
C        1.1.1.1 is directly connected, Loopback0
```

VRF B looks like VRF A, just with Gi0/2. The **global table** only has Loopback0: VRF routes never appear in it.

```
R1#ping 192.168.1.10
.....
R1#ping vrf A 192.168.1.10
!!!!!
R1#ping vrf B 192.168.1.10
!!!!!
```

A plain `ping` uses the **global table**, and the global table has no route to 192.168.1.0/24, so it fails. `ping vrf A` looks up the address in VRF A's table and goes out Gi0/1 to A1. `ping vrf B` looks it up in VRF B's table and goes out Gi0/2 to B1. It's the same address, but two different devices answer. Every tool works this way: `traceroute vrf`, `show ip arp vrf`, `telnet ... /vrf`.

Task 4

```
R1(config)#ip route vrf A 10.10.10.0 255.255.255.0 192.168.1.10
R1(config)#ip route vrf B 10.10.10.0 255.255.255.0 192.168.1.10
```

```
R1#show ip route vrf A static | include 10.10
S        10.10.10.0 [1/0] via 192.168.1.10
R1#ping vrf A 10.10.10.1
!!!!!
R1#ping vrf B 10.10.10.1
!!!!!
A1#ping 1.1.1.1
U.U.U
```

Each static route lives in its own VRF and resolves its next hop in that VRF. So the "same" route points to A1 in VRF A and to B1 in VRF B. A1 can't reach 1.1.1.1: R1 receives the packet on Gi0/1 (VRF A), and VRF A has no route to 1.1.1.1, so R1 drops it and answers with ICMP unreachable (the `U`). That isolation is the whole point of VRFs.

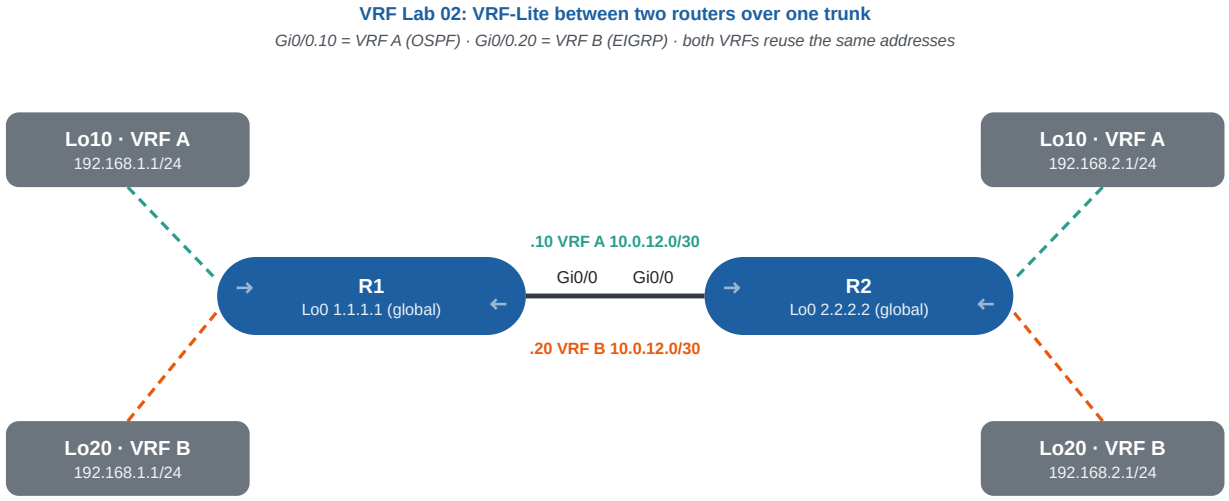
VRF Lab 02: VRF-Lite Across Two Routers

Lab Objective:

The focus of this lab is to extend VRFs between two Cisco IOS routers with VRF-Lite. Additional technologies include 802.1Q sub-interfaces as per-VRF links, OSPF in a VRF, named-mode EIGRP in a VRF, per-VRF neighbor and route verification, and overlapping addresses on the transit link.

Lab Topology:

The lab network topology is illustrated below:



IMPORTANT NOTE

Use IOSv for R1 and R2, connected Gi0/0 to Gi0/0. Each customer site is simulated with a loopback inside the VRF.

VRF-Lite has one rule that shapes the whole design: **every VRF needs its own link** between the routers. One physical cable carries several VRFs by giving each VRF its own 802.1Q sub-interface.

Task 1

On R1 and R2, create VRFs **A** and **B** for IPv4. Configure Loopback0 in the global table (R1 1.1.1.1/32, R2 2.2.2.2/32). Create the site loopbacks from the topology: Lo10 in VRF A and Lo20 in VRF B.

Task 2

Connect the two routers with one sub-interface per VRF on Gi0/0: VLAN 10 for VRF A and VLAN 20 for VRF B. Both VRFs use 10.0.12.0/30 on their link (R1 .1, R2 .2). Verify that each VRF can ping across its own link.

Task 3

Run OSPF process 10 in VRF A on both routers for the link and the site loopback. Use router IDs 10.0.0.1 (R1) and 10.0.0.2 (R2). Make the loopbacks appear with their real /24 mask. Verify the OSPF neighbor and the routes in VRF A.

Task 4

Run named-mode EIGRP **LAB** with autonomous system 20 in VRF B on both routers, for the link and the site loopback. Verify the EIGRP neighbor and the routes in VRF B.

Task 5

Ping between the two VRF A sites and between the two VRF B sites, sourced from the site loopbacks. Verify that the two customers stay separated, and that the global table has learned nothing.

Lab Validation:

The outputs below are trimmed.

Task 1

On R1 (R2 the same, with its own addresses):

```
R1(config)#vrf definition A
R1(config-vrf)#address-family ipv4
R1(config-vrf-af)#exit-address-family
R1(config-vrf)#vrf definition B
R1(config-vrf)#address-family ipv4
R1(config-vrf-af)#exit-address-family
R1(config-vrf)#exit
R1(config)#interface Loopback0
R1(config-if)#ip address 1.1.1.1 255.255.255.255
R1(config-if)#interface Loopback10
R1(config-if)#vrf forwarding A
R1(config-if)#ip address 192.168.1.1 255.255.255.0
R1(config-if)#ip ospf network point-to-point
R1(config-if)#interface Loopback20
R1(config-if)#vrf forwarding B
R1(config-if)#ip address 192.168.1.1 255.255.255.0
```

R2 uses 192.168.2.1/24 on its Lo10 and Lo20. `ip ospf network point-to-point` on Lo10 makes OSPF advertise the loopback as a /24 instead of a /32 (Task 3).

Task 2

```
R1(config)#interface GigabitEthernet0/0
R1(config-if)#no shutdown
R1(config-if)#interface GigabitEthernet0/0.10
R1(config-subif)#encapsulation dot1q 10
R1(config-subif)#vrf forwarding A
R1(config-subif)#ip address 10.0.12.1 255.255.255.252
R1(config-subif)#interface GigabitEthernet0/0.20
R1(config-subif)#encapsulation dot1q 20
R1(config-subif)#vrf forwarding B
R1(config-subif)#ip address 10.0.12.1 255.255.255.252
```

R2 is the same with .2. Each sub-interface is a separate Layer 3 interface with its own VLAN tag, so each VRF gets **its own link** on one cable. A switch between the routers would just need a trunk with VLANs 10 and 20.

```
R1#ping vrf A 10.0.12.2
!!!!!
R1#ping vrf B 10.0.12.2
!!!!!
```

Both pings go to 10.0.12.2, over the same cable, to two different routing tables on R2.

Task 3

```
R1(config)#router ospf 10 vrf A
R1(config-router)#router-id 10.0.0.1
R1(config-router)#network 10.0.12.0 0.0.0.3 area 0
R1(config-router)#network 192.168.1.0 0.0.0.255 area 0
```

R2: `router ospf 10 vrf A`, router ID 10.0.0.2, networks 10.0.12.0/30 and 192.168.2.0/24.

- One OSPF **process per VRF**: `vrf A` on the process line binds it to that VRF's interfaces and routing table
- Set the **router ID by hand**: otherwise the process picks an address from its VRF, and VRFs with overlapping addresses can end up with the same ID on both routers

```
R1#show ip ospf 10 neighbor
Neighbor ID     Pri   State           Dead Time   Address         Interface
10.0.0.2          1   FULL/DR         00:00:35    10.0.12.2       GigabitEthernet0/0.10

R1#show ip route vrf A ospf | include 192.168
O        192.168.2.0/24 [110/2] via 10.0.12.2, 00:00:21, GigabitEthernet0/0.10
```

Plain `show ip ospf neighbor` also works and lists the neighbors of every process, including the VRF ones.

Task 4

```
R1(config)#router eigrp LAB
R1(config-router)#address-family ipv4 unicast vrf B autonomous-system 20
R1(config-router-af)#eigrp router-id 20.0.0.1
R1(config-router-af)#network 10.0.12.0 0.0.0.3
R1(config-router-af)#network 192.168.1.0 0.0.0.255
```

R2 is the same with router ID 20.0.0.2 and 192.168.2.0/24. In named mode, one EIGRP instance holds **one address family per VRF**, each with its own AS number. Classic mode can do the same with `router eigrp 1` + `address-family ipv4 vrf B autonomous-system 20`.

```
R1#show ip eigrp vrf B neighbors
EIGRP-IPv4 VR(LAB) Address-Family Neighbors for AS(20)
           VRF(B)
H   Address                 Interface              Hold Uptime   SRTT   RTO  Q  Seq
0   10.0.12.2               Gi0/0.20                 13 00:00:41    3   100  0  4

R1#show ip route vrf B eigrp | include 192.168
D        192.168.2.0/24 [90/10880] via 10.0.12.2, 00:00:38, GigabitEthernet0/0.20
```

Task 5

```
R1#ping vrf A 192.168.2.1 source Loopback10
!!!!!
R1#ping vrf B 192.168.2.1 source Loopback20
!!!!!
R1#show ip route | include 192.168|10.0.12
R1#
```

Both customers use 192.168.1.0/24, 192.168.2.0/24 and 10.0.12.0/30 at the same time, on the same two routers and the same cable. Each VRF has its own neighbor relationship and its own routes, and the **global table** knows nothing about either. VRF A's routes come from OSPF and VRF B's from EIGRP: each VRF can run whatever routing protocol fits that customer.

The limit of VRF-Lite shows up when you think about growing this: 10 VRFs need 10 sub-interfaces and 10 routing processes on **every** link between routers. That's why large networks carry VRFs with **MPLS L3VPN** (MP-BGP), where the core doesn't need per-VRF configuration.

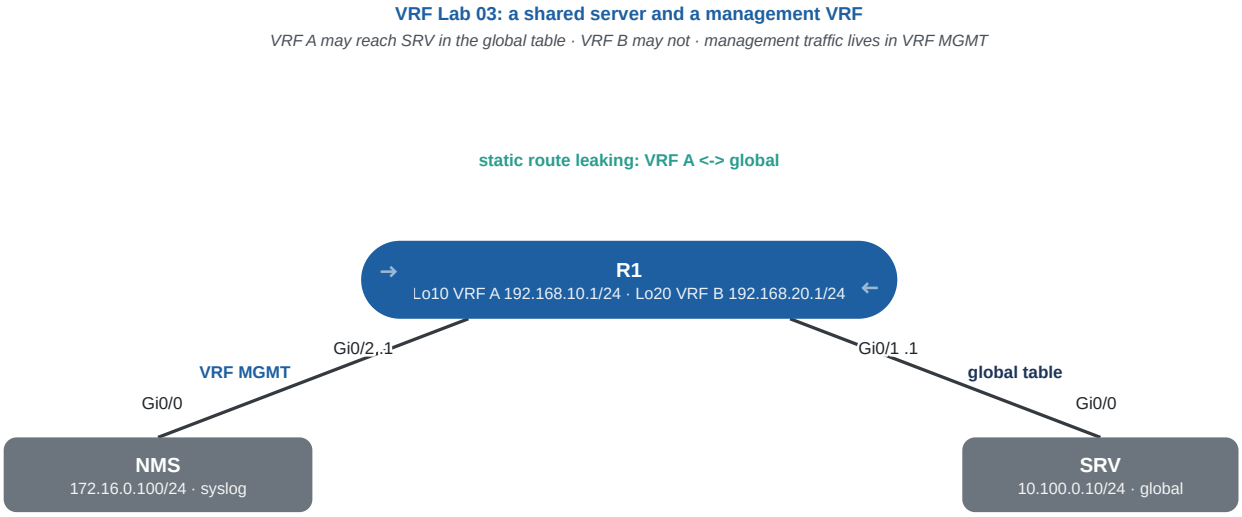
VRF Lab 03: Shared Services and a Management VRF

Lab Objective:

The focus of this lab is to let VRFs talk to the outside in a controlled way. Additional technologies include static route leaking between a VRF and the global table, the limits of route leaking with overlapping addresses, a dedicated management VRF, and VRF-aware management services such as syslog.

Lab Topology:

The lab network topology is illustrated below:



IMPORTANT NOTE

Use IOSv for R1, SRV and NMS. SRV and NMS are routers acting as hosts, each with a default route to R1. SRV sits in R1's **global** table. The two customer sites are loopbacks inside VRFs A and B, and this time they use **different** subnets.

Task 1

On R1, create VRFs **A**, **B** and **MGMT** for IPv4. Configure Lo10 in VRF A (192.168.10.1/24), Lo20 in VRF B (192.168.20.1/24), and Gi0/1 in the global table (10.100.0.1/24). Configure SRV with 10.100.0.10/24 and a default route to R1. Verify that VRF A can't reach SRV yet.

Task 2

Let VRF A reach SRV with static routes only, in both directions. VRF B must still be unable to reach SRV. Verify with pings sourced from the VRF loopbacks.

Task 3

In VRF Lab 02, both customers used 192.168.1.0/24. Explain why the same leaking wouldn't work if VRFs A and B both used 192.168.10.0/24, and what is normally used instead.

Task 4

Put R1 Gi0/2 into VRF **MGMT** with 172.16.0.1/24, and configure NMS with 172.16.0.100/24. Allow SSH to R1 with user `admin` / password `cisco`. Verify that NMS can SSH to R1, and that the management network can't be reached from the global table or from the customer VRFs.

Task 5

Send R1's syslog messages to NMS through VRF MGMT. Generate a log message and verify that it was sent.

Lab Validation:

The outputs below are trimmed.

Task 1

```
R1(config)#vrf definition A
R1(config-vrf)#address-family ipv4
R1(config-vrf-af)#exit-address-family
R1(config-vrf)#vrf definition B
R1(config-vrf)#address-family ipv4
R1(config-vrf-af)#exit-address-family
R1(config-vrf)#vrf definition MGMT
R1(config-vrf)#address-family ipv4
R1(config-vrf-af)#exit-address-family
R1(config-vrf)#exit
R1(config)#interface Loopback10
R1(config-if)#vrf forwarding A
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#interface Loopback20
R1(config-if)#vrf forwarding B
R1(config-if)#ip address 192.168.20.1 255.255.255.0
R1(config-if)#interface GigabitEthernet0/1
R1(config-if)#ip address 10.100.0.1 255.255.255.0
R1(config-if)#no shutdown
```

```
SRV(config)#interface GigabitEthernet0/0
SRV(config-if)#ip address 10.100.0.10 255.255.255.0
SRV(config-if)#no shutdown
SRV(config-if)#exit
SRV(config)#ip route 0.0.0.0 0.0.0.0 10.100.0.1
```

```
R1#ping 10.100.0.10
!!!!!
R1#ping vrf A 10.100.0.10 source Loopback10
.....
```

The global table reaches SRV. VRF A has no route to 10.100.0.0/24, so the ping fails.

Task 2

Two static routes, one per direction:

```
R1(config)#ip route vrf A 10.100.0.0 255.255.255.0 10.100.0.10 global
R1(config)#ip route 192.168.10.0 255.255.255.0 Loopback10
```

Route	Table	Meaning
`ip route vrf A 10.100.0.0 ... 10.100.0.10 global`	VRF A	To reach SRV, use next hop 10.100.0.10, and look that next hop up in the **global** table
`ip route 192.168.10.0 ... Loopback10`	Global	To reach VRF A's site, send it out Loopback10, which is in VRF A

Each route **crosses** from one table into the other: that's **route leaking**. The `global` keyword at the end of a VRF static route is what lets the next hop live outside the VRF. In the other direction, a global route can point at an interface that sits in a VRF.

```
R1#show ip route vrf A static | include 10.100
S        10.100.0.0/24 [1/0] via 10.100.0.10
R1#ping vrf A 10.100.0.10 source Loopback10
!!!!!
R1#ping vrf B 10.100.0.10 source Loopback20
.....
```

Only VRF A got a route, so only VRF A can reach SRV. That's what **shared services** look like: a DNS, NTP or update server that several customers may use, without those customers seeing each other.

Task 3

Leaking works in both directions only if every leaked prefix is **unique in the global table**. If VRFs A and B both used 192.168.10.0/24, the global table would need two routes for the same prefix, one to Lo10 and one to Lo20. It can hold only one, so SRV's replies would always go to the same customer.

With overlapping customer addresses, the shared service is reached through **NAT** instead. Each VRF's traffic is translated to a unique address before it reaches the shared network (VRF-aware NAT: `ip nat inside source ... vrf <name>`). In MPLS networks, the same leaking is done more cleanly with BGP **route targets** (import and export).

Task 4

```
R1(config)#interface GigabitEthernet0/2
R1(config-if)#vrf forwarding MGMT
R1(config-if)#ip address 172.16.0.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#exit
R1(config)#ip domain name lab.local
R1(config)#crypto key generate rsa modulus 2048
R1(config)#username admin privilege 15 secret cisco
R1(config)#line vty 0 4
R1(config-line)#login local
R1(config-line)#transport input ssh
```

NMS: 172.16.0.100/24 on Gi0/0 with a default route to 172.16.0.1.

```
NMS#ssh -l admin 172.16.0.1
Password:
R1#
R1#ping 172.16.0.100
.....
R1#ping vrf MGMT 172.16.0.100
!!!!!
```

SSH works without any VRF configuration on the VTY lines: R1 accepts management sessions on interfaces in any VRF. The management network exists **only in VRF MGMT**. User traffic in the global table or in the customer VRFs has no route to it, and a mistake in the production routing can't cut you off from management. That's the reason for a **management VRF** (on many platforms it already exists as `Mgmt-intf` or `Mgmt-vrf` on the dedicated management port).

Task 5

```
R1(config)#logging host 172.16.0.100 vrf MGMT
R1(config)#logging trap informational
R1(config)#interface Loopback99
R1(config-if)#no shutdown
```

```
R1#show logging | include 172.16.0.100
    Logging to 172.16.0.100  (udp port 514, audit disabled,
              link up),
              5 message lines logged,
              VRF Name: MGMT
```

Without `vrf MGMT`, R1 would look up 172.16.0.100 in the **global** table, find no route and send nothing. Every management service needs to know which table to use: `logging host ... vrf`, `ntp server vrf`, `snmp-server host ... vrf`, and `ip tftp source-interface <MGMT interface>` for file copies. This VRF-awareness is the part people most often forget when they introduce a management VRF.